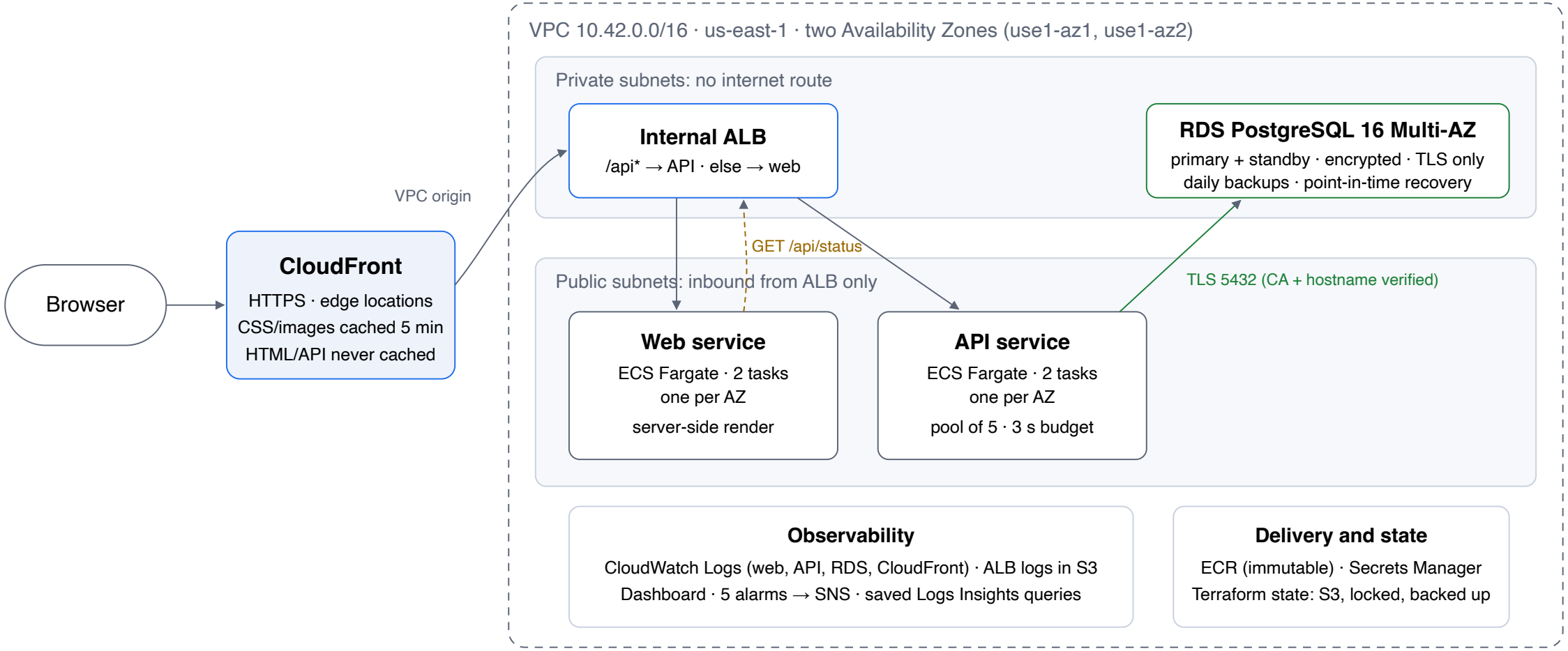
Three-tier Node application on AWS

Continuous delivery architecture: scalable, secure, measured

Hassan Khattak · DevOps portfolio project · October 2026

Deployed on AWS us-east-1, since torn down

Architecture



All resources defined in Terraform (infra/)

Requirements → solution

Requirement	Solution	Evidence
Web and API public, DB private	CloudFront only public entry; internal ALB; private RDS	Tasks, ALB and DB unreachable from the internet (tested)
Everything as code	Terraform with S3 state and locking	Repeat plans clean, in CI too
Instance failures	2 tasks/service across 2 AZs; RDS Multi-AZ	Task loss: 0 failures; DB failover ~17 s
Updates without downtime	Rolling releases; parallel-generation DB upgrade	Releases and DB upgrade: 0 failures
Automated deployment + tests	GitLab CI: tests → plan → build → release	53 unit tests gate the release
Daily backups	RDS automated + PITR; AWS Backup for state	Scheduled snapshot 02:08 UTC; restore drill passed
Central logs	CloudWatch Logs for every tier; ALB logs in S3	Request-ID tracing across web and API
Historical metrics	CloudWatch dashboard and alarms	Alarm caught the failover test's 5xx
CDN by location	CloudFront edge caching for static assets	`Hit` with rising `Age`; dynamic never cached

Security

- **One public entry.** The ALB is internal; only CloudFront's managed security group may reach it.
- **Inbound paths:** ALB → tasks :3000, API → DB :5432. Nothing else.
- **Database:** TLS required by the server; the API verifies the RDS CA and hostname; read-only app login.
- **Secrets:** Secrets Manager, injected by ECS; master password managed by RDS. None in Git, images or Terraform.
- **IAM:** one execution role per service; task roles with no permissions; web cannot read the DB secret.
- **CI:** plan (read-only), release and infra roles. Deploy credentials only on protected `main`.
- **Containers:** Alpine, non-root, read-only filesystem, pinned by digest, scanned on push (0 findings).
- **Encryption at rest:** RDS, S3, ECR.

Failure handling: measured

0
failed requests when a web or API task was killed

45–68 s
until ECS had a healthy replacement

~17 s
of fast 503s during a forced DB failover

0
container restarts during the DB outage

- Sampler inside the VPC: `/` and `/api/status` every 500 ms during each test.
- Requests are bounded (API 3 s, web 5 s) and fail fast with 503; the pool reconnects by itself.
- **Liveness ≠ readiness:** health checks never depend on the DB, so a DB outage doesn't cause a restart storm.

Updates without downtime

Application releases

- Rolling: minimum healthy 100%, maximum 200%.
- New tasks healthy before old ones drain (15 s deregistration, ≤ 20 s graceful shutdown).
- Circuit breaker + public smoke test + automatic rollback.

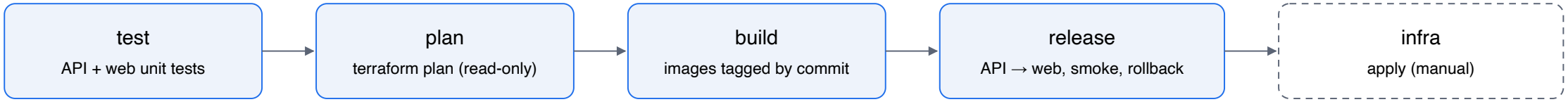
Measured: 0 failed requests

Database engine update (16.14 → 16.15)

- Restore **green** from serving **blue**; upgrade green while blue serves.
- Verify green as the app user over TLS.
- Configuration release moves the API to green; blue stays for rollback.
- Safe because the application is read-only.

Measured: 0 failed of 3,598 requests, rollback tested

Continuous delivery



- **Every branch:** tests, a check that CI is read-only, Terraform plan. **Main:** build and one serialized release job.
- The release waits for *all* checks; only one release runs at a time; image digests come from ECR by the commit tag.
- CI runs in a separate GitLab project on an owned runner; the code repository itself needs no runners.
- AWS access through scoped roles. OIDC is configured, but this GitLab issues no ID tokens, so CI users can only assume those roles.

Operations, backups and observability

Runtime scripts `scripts/runtime/`

- `status.sh`, `probe.sh` (in-VPC check)
- `scale.sh web 3`, `start.sh`, `stop.sh --confirm-outage`
- `teardown.sh --confirm-destroy`

Backups `scripts/backup/`

- RDS daily, 7 days, point-in-time recovery
- State bucket: versioned + AWS Backup daily
- `backup-now.sh`, `restore-drill.sh` (20 min, verified)

Logs

- JSON request logs with request ID (= UUID on the page)
- RDS logs, ALB logs in S3, CloudFront logs

Metrics

- Dashboard: CloudFront, ALB latency/5xx, healthy targets, ECS CPU/memory, RDS
- Alarms: targets < 2, 5xx, CDN error rate, DB CPU

Trade-offs and next steps

Choice made	Why	Next step with more time
Tasks in public subnets, outbound only	No NAT gateway cost; inbound closed by security groups	Private subnets + VPC endpoints
~17 s DB failover window	Managed Multi-AZ, fast-failing requests	RDS Proxy or Aurora
Fixed 2 + 2 tasks, scripted scaling	Predictable cost for this project	ECS Service Auto Scaling
Default CloudFront certificate	No custom domain required	Domain + ACM certificate + WAF
CI users instead of OIDC	Git host issues no ID tokens	OIDC once supported

Not simulated: a full Availability Zone outage; a fresh-account bootstrap end to end.